Digital Backend Controller in Verilog

Samarth Sharma Bhardwaj
Roll Number: 230903

EE619: VLSI System Design
Project 1

Submitted On: March 7, 2026

Submitted to
Prof. Avinash Lahgere, Prof. Chithra

Indian Institute of Technology, Kanpur

1 Description of Design and Problem Statement

The task was to design a backend for a mixed signal IC.

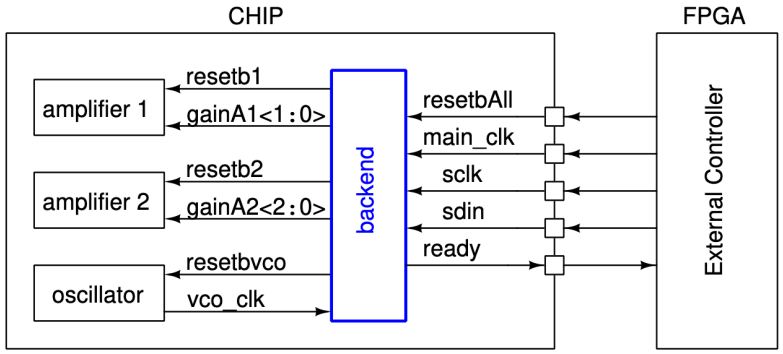

Figure 1: Overall block diagram.

Figure 1: Block Diagram

The chip consists of two amplifiers and an oscillator. The backend has to ensure that these analog modules are initialized correctly and the chip is ready for use. The backend has to execute a start-up sequence for this.

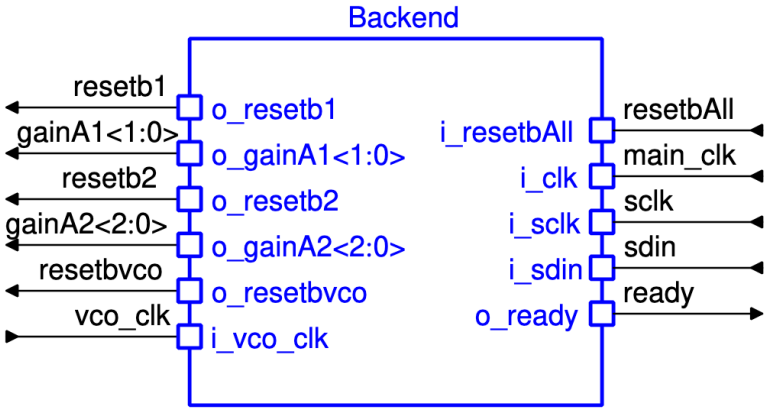

Figure 2: Input output configuration of the backend.

Figure 2: I/O ports

2 File Structure

The entire code was organized in the following files:

- *main.v* - This contains the main module which instantiates other modules and does the appropriate wiring.
- *serial.v* - This contains the module that is responsible for the serial communication between the FPGA and the backend chip.
- *startup.v* - This contains the module that generates appropriate resets for the startup sequence. A Moore type finite state machine (FSM) was used for this task.
- *tb.v* - This contains a comprehensive testbench that was generated by Claude (AI assistant) to check the functionality of the code.

These are also uploaded on this GitHub repository.

3 Serial Communication Module

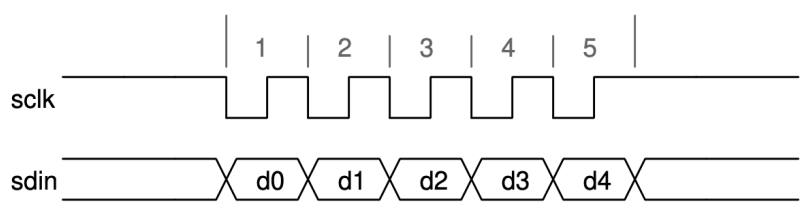

Figure 3: Serial data transmission using sclk and sdin lines

The FPGA will sent the data at the falling edge of sclk. The backend has to receive these at the rising edge of sclk. The data corresponding to the first sclk cycle is d0, second sclk cycle is d1 and so on.

Designed module-

```
module serial_comm (
    input i_sclk,
    input i_sdin,
    input i_reset, // this will be an active low reset
    output reg [1:0] o_gainA1,
    output reg [2:0] o_gainA2,
    output reg ready_serial
);
```

Inputs-

```
i_sclk, i_sdin, i_reset
```

These are the clock for serial communication, the input port and reset for this module respectively.

Outputs-

```
[1:0] o_gainA1, [2:0] o_gainA2, ready_serial
```

The first two are the gains of the amplifiers respectively. The last output is a signal that is used to know if the serial communication is complete or not. It is set once the communication is complete.

The entire code is on the github repository in the file (*serial.v*). The module updates the *ready_serial* flag in the clock cycle *after* the fifth clock cycle for ease of design. The reset in this block was chosen to be asynchronous active low. Comments have been added in the code to explain main functionality.
The *final_stop* flag ensures that the gains are only assigned once.
A counter is used to wait 5 clock cycles to ensure serial communication completes. Due to the use of non-blocking assignments in sequential logic, the condition on the counter is set to 4.
A shift register - type logic was used for this module, and finally the amplifier gains are extracted from the data received.

4 Startup Sequence design using FSM

A Moore Finite State Machine was used for this purpose. A Mealey machine could also have been used and it would have saved the final ready state, but since the number of flip flops would have still stayed the same (i.e. 3), the decision of Moore machine was taken to ensure output stability even if input is unstable since outputs are fixed at states and do not change or depend on inputs/transitions.

An 8-state Moore FSM was designed with the following states-

1. **RST** - This is when reset is 0 (i.e. activated since it is active low). All outputs are pulled to 0 and regs are reset. We leave this set is reset is deactivated (i.e set).

2. **WAIT_SERIAL** - Serial communication happens in this state. This module is **not** instantiated in this module, and is done in the main file instead. Instead, the flag which shows completion of communication and is tracked. If that flag is set, we proceed to the next state. More details on the flag will come later.

3. **WAIT_2CYC** - Waits two clock cycles. A counter is used for this purpose. The delay counters are reset automatically when the FSM transitions between states. This ensures that each delay period starts from zero when entering the corresponding wait state.

4. **SET_VCO** - The reset of the voltage controlled oscillator is deactivated (i.e. set).

5. **WAIT_10A** - This waits for 10 clock cycles. Again, a counter is used. To ensure that the outputs are held (since we are doing all this in a combinational block with default values), we explicitly set the reset again to ensure the value holds. This would be done in all the states that follow due to the same reason.

6. **SET_AMPS** - Sets the resets of the amplifiers to high to deactivate them.

7. **WAIT_10B** - Again 10 cycles are waited. Similar counter logic used. It is possible to reuse the old state but that has not been done for simplicity. Just like before, all previous outputs are set to ensure they hold.

8. **DONE** - This is the final state after which the backend has successfully configured the mixed signal chip. The ready signal (along with ALL other resets) is set.

I have also used a simple 2-DFF based synchronizer to account for clock domain crossing. The signal *ready_serial* comes from the clock *i_sclk* of the serial communicator while the others are on the main high frequency *i_clk*. For this, a synchronizer is used which is standard for solving clock domain crossing problems. The entire code is available in the github repo in the file '*startup.v*'.

5 Final wiring and top module

All the blocks were wired in the file *main.v*. The top module description is given below-

```
    module backend(
    input i_resetbAll, i_clk, i_sclk, i_sdin, i_vco_clk,
    output o_resetb1, o_resetb2, o_resetbvco, o_ready,
    output [1:0] o_gainA1,
    output [2:0] o_gainA2
);
```

This is a pretty straightforward module that just instantiates the startup (FSM) and serial communication modules, outputting the amplifier gains and all appropriate resets, alongside the final ready signal.

6 Verification

A testbench was generated using Claude's AI assistant which checked the functionality of the code under all cases. The design passed all the test cases. The final log is given below-

```
=== TEST 1: Reset State ===
  PASS: All outputs 0 during reset
TEST 1 done

=== TEST 2: Normal startup - gainA1=2'b10, gainA2=3'b101 ===
  [t=2788000] Checking all outputs after full sequence:
  PASS o_resetbvco=1
  PASS o_resetb1=1
  PASS o_resetb2=1
  PASS o_ready=1
  PASS o_gainA1=10
  PASS o_gainA2=101
TEST 2 done

=== TEST 3: Outputs hold in DONE state ===
  PASS: All outputs held after 50 extra cycles
TEST 3 done

=== TEST 4: Re-reset behavior ===
  PASS: FSM outputs cleared to 0
  PASS: Gain outputs cleared to 0
TEST 4 done

=== TEST 5: All-ones gains - gainA1=2'b11, gainA2=3'b111 ===
  [t=5813000] Checking all outputs after full sequence:
  PASS o_resetbvco=1
  PASS o_resetb1=1
  PASS o_resetb2=1
  PASS o_ready=1
  PASS o_gainA1=11
  PASS o_gainA2=111
TEST 5 done

=== TEST 6: All-zero gains - gainA1=2'b00, gainA2=3'b000 ===
  [t=8588000] Checking all outputs after full sequence:
  PASS o_resetbvco=1
  PASS o_resetb1=1
  PASS o_resetb2=1
  PASS o_ready=1
  PASS o_gainA1=00
```

```
  PASS o_gainA2=000
TEST 6 done

=== TEST 7: Mid-sequence reset ===
  Asserting reset mid-sequence (VCO released, amps not yet) at t=11153000
  PASS: All outputs cleared after mid-sequence reset
TEST 7 done

=== TEST 8: Recovery after mid-sequence reset ===
  [t=13928000] Checking all outputs after full sequence:
  PASS o_resetbvco=1
  PASS o_resetb1=1
  PASS o_resetb2=1
  PASS o_ready=1
  PASS o_gainA1=01
  PASS o_gainA2=010
TEST 8 done

=== TEST 9: Alternating bits - gainA1=2'b01, gainA2=3'b101 ===
  [t=16703000] Checking all outputs after full sequence:
  PASS o_resetbvco=1
  PASS o_resetb1=1
  PASS o_resetb2=1
  PASS o_ready=1
  PASS o_gainA1=01
  PASS o_gainA2=101
TEST 9 done

=== ALL TESTS COMPLETE ===
```

7 AI assistant help declaration

I declare that I have used Claude's AI assistant for help throughout the project.
The entire chat log is given on this link- Chat Log.